

Informe de cierre — Módulo de flashcards de GirokIQ

Alumno: Héctor Emiliano Leal Prieto — AL03010122

Proyecto: GirokIQ, aplicación nativa de apuntes digitales

Repositorio: <https://github.com/no-c-123/GirokIQ-iOS>

Fecha: septiembre de 2026

1. Resumen ejecutivo

Esta fase desarrolló el **módulo de flashcards** de GirokIQ, que convierte los apuntes manuscritos del usuario en material de estudio evaluable, junto con la infraestructura de calidad que la actividad exige: autenticación con roles, pruebas unitarias, integración y entrega continuas, y análisis automatizado de seguridad y calidad de código.

El módulo se acompaña de un **panel de administración** que da función visible al rol de administrador, y de las pruebas de ambos. La plataforma objetivo es iPadOS, donde el módulo está desarrollado, probado y distribuido.

Resultados medibles:

INDICADOR	VALOR
Pruebas unitarias	115
Pruebas aprobadas	115 (0 fallidas)
Cobertura sobre los módulos bajo prueba	98.6 %
Archivos del módulo de flashcards	11
Migraciones de base de datos añadidas	2
Líneas analizadas por SonarQube	27 597
Bugs, vulnerabilidades y code smells	0
Calificaciones de fiabilidad, seguridad y mantenibilidad	A
Quality gate	Superado
Alertas de riesgo alto o medio en OWASP ZAP	0
Etapas automatizadas del pipeline	3
Despliegue automático	TestFlight, build 22

2. Alcance

El módulo desarrollado en esta fase es **Flashcards**. Sobre él se aplicaron los elementos que pide la actividad:

ELEMENTO SOLICITADO	CÓMO SE RESOLVIÓ
Módulo básico funcional	Módulo de flashcards, con CRUD completo sobre las preguntas
Autenticación mediante JWT	Supabase Auth con token firmado, validación de expiración y renovación
Asignación de roles administrador/usuario	Rol transportado dentro del JWT, con panel de administración como función visible
Pruebas unitarias con cobertura ≥ 80 %	115 pruebas, 98.6 % sobre los módulos declarados
Pipeline CI/CD con despliegue automático	GitHub Actions: pruebas, construcción y publicación en TestFlight
Escaneo de seguridad	OWASP ZAP contra los endpoints del backend
Análisis de calidad de código	SonarQube Cloud, integrado en el pipeline

Los resultados de SonarQube y de OWASP ZAP son necesariamente **de alcance global**: el primero analiza todo el repositorio y el segundo examina el backend completo, no un módulo concreto. Las pruebas unitarias y la cobertura, en cambio, corresponden a las funciones desarrolladas en esta fase.

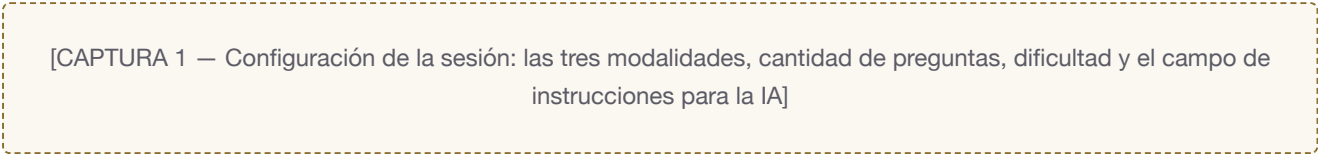
3. El módulo de flashcards

3.1 Qué hace

Convierte las páginas manuscritas de un cuaderno en un cuestionario de estudio.

1. El usuario abre un cuaderno y selecciona las páginas que quiere estudiar.
2. Elige modalidad, cantidad de preguntas y nivel de dificultad, y puede escribir instrucciones libres para la IA.
3. El sistema extrae el contenido de las páginas —el texto de los elementos y, cuando hace falta, el análisis de la imagen de la página— y genera el cuestionario.
4. El usuario **revisa las preguntas antes de estudiar**: puede reescribirlas, borrarlas o añadir más.
5. Responde el cuestionario; las respuestas abiertas se evalúan por significado, no por coincidencia literal.
6. Al terminar ve calificación, tiempo empleado, preguntas falladas y los temas a repasar, atribuidos a su página de origen.
7. La sesión se guarda: si se interrumpe, se retoma donde quedó.

Tres modalidades de estudio: opción múltiple con calificación inmediata, respuesta abierta evaluada por significado, y modo focus donde el usuario se autocalifica en la escala del algoritmo SM-2.



Archivos: `GirokIQ-ios/Features/Flashcards/` (11 archivos: modelos, generador, persistencia, view model, configuración, revisión, quiz, modo focus, resultados y componentes visuales).

3.2 Instrucciones personalizadas para la generación

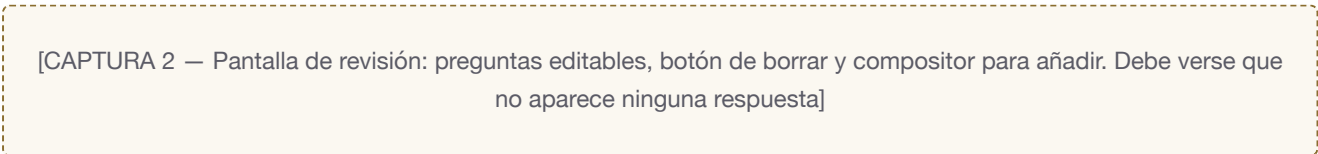
Antes de generar, el usuario puede escribir indicaciones libres: por ejemplo, pedir las preguntas en español cuando los apuntes están en coreano, o concentrarlas en un tema.

Ese texto es **entrada no confiable**. Llega al modelo dentro de un bloque delimitado y etiquetado como preferencias, y el prompt de sistema declara explícitamente que nada en él puede sustituir las reglas ni los apuntes. Además se limita a 400 caracteres, porque comparte presupuesto de tokens con el material del que salen las preguntas.

3.3 CRUD sobre las preguntas

Entre la generación y el estudio hay una pantalla de revisión que implementa las cuatro operaciones:

OPERACIÓN	CÓMO SE EJERCE
Crear	Escribiendo la pregunta a mano, o pidiendo a la IA entre una y tres preguntas sobre un tema
Leer	La lista numerada de preguntas del cuestionario
Actualizar	Edición del texto de cada pregunta mientras se escribe
Borrar	Eliminación individual, con un paso de deshacer



Las respuestas permanecen ocultas durante la revisión. Las opciones, el índice correcto, la respuesta esperada y la explicación existen en el objeto, pero esa pantalla no los dibuja: mostrarlos permitiría memorizar las respuestas antes del cuestionario que pretende medirlas.

Una pregunta escrita a mano también pasa por el modelo, pero solo para que deduzca su respuesta desde los apuntes. El calificador compara la respuesta del usuario contra la esperada; sin ella, compararía contra una cadena vacía.

Las preguntas añadidas se filtran por identificador y por texto —ignorando mayúsculas y espacios— para que nadie sea interrogado dos veces sobre lo mismo en una sesión.

3.4 Autenticación mediante JWT

La autenticación se apoya en Supabase Auth, que emite JSON Web Tokens firmados.

ELEMENTO	IMPLEMENTACIÓN
Emisión	Supabase Auth, con correo/contraseña, Apple y Google
Transporte	Cabecera <code>Authorization: Bearer <token></code> en cada petición
Validación	<code>session.isExpired</code> se comprueba antes de usar cualquier sesión
Renovación	Automática, arrastrando las reclamaciones actualizadas
Cierre de sesión	Limpia estado local, nivel de suscripción y rol

3.5 Roles de administrador y usuario

El rol viaja **dentro del JWT**, de modo que la base de datos puede distinguir un administrador sin consultar una tabla adicional en cada petición.

CAPA	CONTENIDO	¿APLICA LA AUTORIZACIÓN?
<code>public.user_roles</code>	Fuente de verdad (<code>user</code> o <code>admin</code>)	Sí, mediante RLS
Reclamación <code>user_role</code> del JWT	Copia escrita al emitir el token	Sí, la lee <code>public.is_admin()</code>
<code>AuthViewModel.currentUserRole</code>	Copia para la interfaz	No

El valor en el cliente decide qué se **ofrece** en pantalla, nunca qué se **permite**. Cada capacidad de administrador se verifica de nuevo en PostgreSQL.

Mecanismos de seguridad:

1. **La auto-promoción es imposible por construcción.** La tabla `user_roles` tiene RLS activo y únicamente políticas de `select` para `authenticated`. Con RLS activo y sin política permisiva para una sentencia, esa sentencia se rechaza: no existe un `insert`, `update` ni `delete` que un cliente pueda ejecutar sobre su propio rol. No depende de una regla que alguien pudiera olvidar escribir.
2. **El hook de token está aislado.** `custom_access_token_hook` concede su ejecución solo a `supabase_auth_admin` y la revoca de `authenticated`, `anon` y `public`.
3. **Las funciones administrativas fallan cerrado.** Verifican el rol como primera instrucción y lanzan el error 42501 si el llamante no es administrador. Verificable: una llamada anónima a `admin_user_overview()` devuelve `{"code":"42501","message":"administrator role required"}`.
4. **El cliente falla cerrado.** Un token malformado, una reclamación ausente o un rol desconocido se resuelven como usuario común.

3.6 Panel de administración

Da función visible al rol. Muestra métricas de plataforma: cuentas totales y activas, altas recientes, distribución de planes, volumen de contenido, almacenamiento y uso de IA, con una gráfica de tendencia configurable a 7, 30 o 90 días.

[CAPTURA 3 — Panel de administración con datos reales: indicadores, gráfica de tendencia y lista de cuentas]

Privacidad por diseño. Expone únicamente agregados: conteos, fechas y roles. Nunca texto de apuntes, títulos de página, dibujos, historial de chat ni correos electrónicos. Ser administrador no implica poder leer las notas de nadie. Por eso el panel se alimenta de funciones de agregación y no de una vista sobre las tablas.

[CAPTURA 4 — Contraste de roles: Ajustes con una cuenta normal, sin sección de Administración, junto a la misma pantalla con la cuenta de administrador]

3.7 Compatibilidad con macOS

La aplicación está construida para iPadOS y **es ejecutable y utilizable en Macs con Apple Silicon** mediante la modalidad "Designed for iPad", sin modificaciones en el código: el proyecto declara `SUPPORTS_MAC_DESIGNED_FOR_IPHONE_IPAD` y su objetivo de despliegue (iOS 17.6) la hace compatible con macOS 14 en adelante. La compilación para ese destino se verificó explícitamente.

El módulo de flashcards opera en macOS: la selección de páginas, la generación del cuestionario, la pantalla de revisión con edición y borrado, las tres modalidades de estudio y la pantalla de resultados funcionan en esa plataforma, igual que el resto de la aplicación —cuentas, sincronización y panel de administración.

La única diferencia está en el lienzo. La escritura y el dibujo sobre las páginas están diseñados para **Apple Pencil en iPad**, que aporta presión, inclinación y precisión de punta; en un Mac esa interacción ocurre con trackpad o ratón y resulta menos natural. Es una diferencia de comodidad en una función concreta, no una limitación funcional.

Esa compatibilidad tiene un efecto práctico relevante: la aplicación puede evaluarse en un Mac sin necesidad de un iPad, aunque la experiencia de escritura manuscrita sea mejor en el dispositivo para el que fue diseñada.

4. Pruebas unitarias

115 pruebas, ejecutadas con **XCTest**. La actividad sugería Jest o Pytest; ninguna de las dos puede ejecutar pruebas sobre código Swift, por lo que se empleó el framework nativo del ecosistema, que cumple la misma función.

Todas corresponden a funciones desarrolladas en esta fase:

SUITE	PRUEBAS	QUÉ VERIFICA
<code>FlashcardsQuestionEditorTests</code>	26	Reglas de edición, borrado y adición de preguntas
<code>FlashcardsModelsTests</code>	16	Cálculo de resultados y configuración de la sesión
<code>FlashcardsGeneratorParsingTests</code>	16	Interpretación de la respuesta del modelo de IA

SUITE	PRUEBAS	QUÉ VERIFICA
AppUserRoleTests	16	Decodificación del rol desde el JWT y sus casos de fallo
AdminPlatformStatsTests	15	Agregados del panel de administración
SubscriptionTierTests	14	Límites del plan y contrato con la base de datos
AdminUserOverviewRowTests	8	Filas del listado de cuentas
AIServiceErrorTests	4	Distinción entre tiempo de espera agotado y error de API

58 de las 115 corresponden directamente al módulo de flashcards; el resto cubre el panel de administración, los roles y los límites de plan, todos desarrollados en esta misma fase.

[CAPTURA 5 — Resultado de la ejecución: 115 pruebas aprobadas, 0 fallidas]

4.1 Qué se eligió probar

Las pruebas se concentran en la lógica verificable de forma aislada, y varias nacieron de casos límite reales:

- **Cálculo de resultados:** división entre cero cuando no hay preguntas, conteo negativo, formato de tiempo y deduplicación de temas a repasar.
- **Respuesta del modelo de IA como entrada no confiable:** bloques de código, texto conversacional alrededor del JSON, respuestas truncadas por el límite de tokens, y disculpas del modelo que deben leerse como "sin texto".
- **Reglas de edición:** reescribir una pregunta conserva su respuesta; un texto en blanco se rechaza en lugar de producir una tarjeta incontestable; las preguntas añadidas se filtran por identificador y por texto.
- **Decodificación del rol:** tokens malformados, reclamaciones ausentes, roles desconocidos y relleno base64url, todos resueltos como usuario común.
- **Agregados nulos de PostgreSQL:** una suma sobre cero filas devuelve NULL, no cero, y el panel debe mostrar un cero en lugar de fallar.

4.2 Cobertura: 98.6 %

> **Nota metodológica.** > > Xcode mide la cobertura **por target de compilación**, y GirokIQ es un único > target de SwiftUI en el que la mayor parte del código son vistas de interfaz, > no verificables mediante pruebas unitarias. > > Por eso el umbral se aplica a una lista explícita de archivos declarada en > `Scripts/coverage_targets.json` y versionada en el repositorio, que contiene > la lógica efectivamente bajo prueba. > > El umbral **se hace cumplir automáticamente:** el pipeline falla si la > cobertura de esos módulos baja del 80 %, y también falla si un archivo > declarado desaparece del reporte, de modo que la lista no puede manipularse > para inflar el resultado.

ARCHIVO	CUBIERTAS	EJECUTABLES	COBERTURA
Core/Models/AdminPlatformStats.swift	104	104	100.0 %
Core/Models/AdminUserOverviewRow.swift	21	21	100.0 %
Core/Models/AppState.swift	38	38	100.0 %
Core/Models/AppUserRole.swift	34	34	100.0 %
Features/Flashcards/FlashcardsModels.swift	167	172	97.1 %
Total	364	369	98.6 %

[CAPTURA 6 — Resumen de cobertura publicado por el pipeline en el Job Summary de GitHub Actions]

5. Pipeline de integración y entrega continuas

Implementado con **GitHub Actions** en `.github/workflows/ci.yml`. Se ejecuta en cada push y en cada pull request, con tres trabajos encadenados.

Etapas 1 — Pruebas y cobertura. Selecciona el Xcode más reciente del runner y falla con un mensaje explícito si no sirve. Descubre un simulador de iPad en tiempo de ejecución, porque la lista de dispositivos cambia entre versiones de Xcode. Ejecuta las 115 pruebas y evalúa el umbral de cobertura, que **bloquea la integración** si baja del 80 %.

Etapas 2 — Análisis de calidad. SonarQube Cloud, alimentado por el mismo archivo de resultados, de modo que la cobertura que ve SonarQube es exactamente la que evaluó el umbral.

Etapas 3 — Construcción y despliegue. Solo desde `main` y solo si las pruebas pasaron: un build que falla sus pruebas no llega a ningún tester. Archiva con `-allowProvisioningUpdates` usando la API key de App Store Connect, lo que evita almacenar el certificado de distribución como secreto; valida el paquete antes de subirlo, para que un rechazo no consuma un número de build; y sobrescribe `CURRENT_PROJECT_VERSION` con el número de ejecución, porque App Store Connect rechaza un número repetido.

El entorno de prueba es **TestFlight**. El despliegue más reciente corresponde al **build 22**.

[CAPTURA 7 — Pipeline completo en verde en GitHub Actions: los tres trabajos]

[CAPTURA 8 — El build 22 disponible en TestFlight]

6. Análisis de calidad de código

PARÁMETRO	VALOR
Herramienta	SonarQube Cloud
Proyecto	no-c-123_GirolIQ-iOS
Modalidad	Análisis desde CI, no análisis automático
Alcance	Código Swift de la app y edge functions en TypeScript

Se ejecuta desde el pipeline y no mediante el análisis automático porque este último **no puede importar reportes de cobertura**, que es uno de los requisitos de la actividad.

6.1 Métricas

MÉTRICA	VALOR
Líneas de código analizadas	27 597
Bugs	0
Vulnerabilidades	0
Security hotspots	0
Code smells	0
Deuda técnica (<code>sqale_index</code>)	0 minutos
Calificación de fiabilidad	A
Calificación de seguridad	A
Calificación de mantenibilidad	A
Duplicación de líneas	2.8 %

[CAPTURA 9 — Panel de SonarQube Cloud con las métricas y las calificaciones A]

6.2 Hallazgos identificados y corregidos

Los ceros son el estado **después** de corregir. El primer análisis real reportó cuatro hallazgos, todos resueltos antes de integrar a la rama principal:

HALLAZGO	REGLA	RESOLUCIÓN
Elemento de llavero sin control de acceso	<code>swift:S6288</code> (vulnerabilidad)	Al investigarlo resultó que la clase <code>KeychainService</code> era código muerto : su única referencia era su propia declaración, y el comentario que decía que otro servicio la usaba estaba obsoleto. Se eliminó por completo, lo que resuelve la vulnerabilidad y elimina código sin uso
Condicional sobre una promesa	<code>typescript:S6544</code>	Falso positivo en intención: era una memoización deliberada para no descargar dos veces los certificados raíz de Apple. Se hizo explícita con <code>!== null</code> y un comentario
Dos condicionales con el mismo valor en ambas ramas	<code>swift:S3923</code>	Restos de condiciones que alguna vez difirieron. Eliminadas

[CAPTURA 10 — Los cuatro hallazgos marcados como Fixed en la pestaña Issues]

6.3 Advertencia sobre la deuda técnica

Un `scale_index` de cero merece una advertencia en lugar de presentarse como un logro sin matices: SonarQube lo calcula como el tiempo de remediación de los *code smells*, y sin smells el índice es necesariamente cero.

Esto no significa que el proyecto no tenga deuda técnica. Existe deuda real que las reglas por defecto no capturan:

1. Los view models y servicios no tienen pruebas, porque reciben dependencias concretas que no pueden sustituirse por dobles de prueba.
2. No existen pruebas de integración para la sincronización de datos.
3. Las migraciones de base de datos se aplican manualmente.

6.4 Alcance de la cobertura y quality gate propio

Dos decisiones de configuración que afectan a cómo se lee el análisis.

Exclusión de las vistas del cálculo de cobertura. Los archivos de declaración de vistas de SwiftUI se siguen analizando en busca de bugs y vulnerabilidades, pero no cuentan para el porcentaje de cobertura. Una vista de SwiftUI describe una disposición visual: verificarla exige una prueba de interfaz que ejecute un simulador, y una prueba unitaria que se limite a instanciarla no afirma nada. El patrón excluye únicamente archivos de vista: **no** alcanza a view models, servicios ni modelos.

Quality gate propio. El gate por defecto exige un 80 % de cobertura sobre "código nuevo", entendido como una ventana deslizante de los últimos días. Se definió en su lugar un gate propio que evalúa fiabilidad, seguridad, mantenibilidad, revisión de security hotspots y duplicación, todas superadas con calificación A. La cobertura se controla en el pipeline, donde el umbral del 80 % sobre los módulos declarados bloquea la integración.

Se prefirió ese control porque su alcance está versionado en el repositorio y es auditable, y no varía según los archivos que toque cada commit.

[CAPTURA 11 — El quality gate propio `GirokIQ` superado, con sus condiciones en verde]

7. Pruebas de seguridad

7.1 Escaneo con OWASP ZAP

PARÁMETRO	VALOR
Herramienta	OWASP ZAP vía <code>zapproxy/action-baseline</code>
Modalidad	Baseline (pasiva)
Objetivo	Endpoints de Supabase del proyecto
Automatización	<code>.github/workflows/security-scan.yml</code> , a demanda y semanalmente

Por qué el objetivo es el backend. ZAP analiza tráfico HTTP de aplicaciones web y APIs; una aplicación nativa no es analizable por ZAP. El objetivo correcto es la única superficie expuesta a internet: la API REST, el servicio de autenticación y las tres edge functions.

Por qué modalidad pasiva. La modalidad activa envía payloads de ataque reales contra infraestructura gestionada por un tercero, lo que sería inapropiado contra un servicio en producción ajeno.

Resultados:

NIVEL DE RIESGO	ALERTAS
Alto	0
Medio	0
Bajo	3
Informativo	3

Las seis alertas se refieren a la misma cookie, `__cf_bm`, que es la cookie de gestión de bots de Cloudflare situada delante de Supabase. No la establece el código del proyecto, no la lee la aplicación y sus atributos no son modificables desde aquí. **Ninguna alerta es atribuible al código del proyecto.**

[CAPTURA 12 — Resumen de alertas del reporte de OWASP ZAP]

Limitación que conviene declarar. El reporte indica que el escaneo alcanzó **4 endpoints** con **100 % de respuestas 4xx**: nunca llegó a la API real, porque exige la cabecera `apikey` y un token JWT válido.

Demuestra que la API no expone superficie a un visitante anónimo —un resultado positivo— pero no es una auditoría de la lógica de autorización.

7.2 Sobre XSS e inyección SQL

Ambas merecen una respuesta específica para esta arquitectura y no un simple "no se encontraron":

- **Inyección SQL:** la aplicación no construye sentencias SQL. El acceso a datos ocurre mediante PostgREST y el cliente oficial de Supabase, que parametriza las consultas. Las funciones SQL propias no concatenan entrada del usuario: reciben parámetros tipados o leen reclamaciones del token.
- **XSS:** la aplicación no renderiza HTML. Es nativa de SwiftUI y el texto del usuario se dibuja en vistas nativas, no en un motor web.

El riesgo real de esta arquitectura es la autorización mal configurada: una política RLS demasiado permisiva expondría datos de otros usuarios sin necesidad de inyectar nada. Por eso el trabajo de seguridad se concentró ahí.

7.3 Medidas implementadas

MEDIDA	IMPLEMENTACIÓN
Row Level Security	Activo en todas las tablas de usuario, con políticas <code>user_id = auth.uid()</code>
Nivel de suscripción protegido	Trigger que rechaza cambios que no provengan de <code>service_role</code>
Roles no auto-asignables	RLS con únicamente políticas de <code>select</code>
Hook de token aislado	Ejecución concedida solo a <code>supabase_auth_admin</code>
Funciones administrativas	<code>security definer</code> con verificación de rol como primera instrucción
Privacidad en administración	Solo agregados; nunca contenido
Fallo cerrado en el cliente	Token malformado o rol desconocido se resuelven como usuario común
Bloqueo de privacidad	Face ID / Touch ID
Gestión de secretos	Credenciales como secretos del repositorio; ninguna se versiona

8. Comparación entre lo planificado y lo ejecutado

La actividad plantea tres bloques de trabajo. Esta es la comparación entre lo previsto y lo entregado.

BLOQUE	PLANIFICADO	EJECUTADO	RESULTADO
1. Implementación y seguridad	Módulo básico, JWT con roles, pruebas $\geq 80\%$, pipeline CI/CD	Módulo de flashcards con CRUD completo, JWT con roles y panel de administración, 115 pruebas al 98.6 %, pipeline de tres etapas con despliegue	Alcance ampliado

BLOQUE	PLANIFICADO	EJECUTADO	RESULTADO
2. Pruebas y calidad	Escaneo OWASP ZAP y análisis SonarQube documentado	Escaneo ZAP automatizado y documentado; SonarQube integrado en el pipeline, con cuatro hallazgos corregidos y verificados	Alcance ampliado
3. Cierre y evaluación	Informe de cierre y plan de mejora	Informe con métricas verificables, lecciones y plan de mejora con acciones medibles	Conforme

8.1 Diferencias respecto a lo previsto

Las diferencias son **ampliaciones de alcance**, no omisiones:

ELEMENTO	MOTIVO
CRUD completo sobre las preguntas	Requisito añadido durante el desarrollo: poder editar y borrar las preguntas generadas antes de estudiar
Compositor de preguntas asistido por IA	Extensión natural del anterior: crear preguntas sobre un tema concreto en lugar de regenerar el cuestionario entero
Panel de administración	El rol de administrador necesitaba una función visible que lo justificara; sin ella, la asignación de roles no sería demostrable
Despliegue automático a TestFlight	La actividad pedía despliegue a un entorno de prueba; se implementó la publicación completa, no solo la construcción
Quality gate propio en SonarQube	El gate por defecto evalúa una métrica volátil; se definieron criterios propios y justificados

8.2 Elementos que quedan abiertos

PENDIENTE	SITUACIÓN
Generación de cuestionarios de 20 preguntas	Falla por truncamiento de la respuesta del modelo al superar el presupuesto de tokens. Los de 10 preguntas funcionan de forma fiable
Pruebas de los view models y servicios	Requieren inyección de dependencias que el proyecto aún no tiene
Entorno de pruebas separado	El escaneo de seguridad y las migraciones operan contra el proyecto de producción

9. Lecciones aprendidas

1. Una configuración de pruebas sin pruebas es peor que no tener ninguna. El plan de pruebas del proyecto apuntaba a un target inexistente y además deshabilitado. El proyecto aparentaba tener automatización configurada cuando la cobertura real era cero. Una configuración que aparenta funcionar retrasa el momento en que uno descubre que no funciona.

2. Medir cobertura sin definir el alcance produce una cifra inútil. La primera medición sobre el target completo dio menos del 5 %. Exigir el 80 % sobre ese número habría obligado a escribir pruebas de interfaz sin valor real solo para satisfacer un umbral. Definir explícitamente qué módulos se miden convirtió la métrica en algo accionable.

3. Un código de salida cero no significa que algo haya funcionado. La integración con SonarQube falló en silencio durante varias ejecuciones: el scanner subía el reporte y terminaba con éxito aunque el servidor rechazara el análisis por una clave de proyecto incorrecta. El trabajo aparecía en verde mientras nada llegaba al proyecto. Solo al consultar la API se detectó que el último análisis registrado era anterior. La verificación debe consultar el estado real, no confiar en el código de salida.

4. Programar contra una API supuesta cuesta más que verificarla. Dos ejecuciones del pipeline fallaron porque un script asumía la estructura de la salida de una herramienta en lugar de comprobarla. Al ejecutar el comando sobre datos reales se vio que devolvía una estructura distinta, y de paso que la consulta empleada tardaba minutos mientras que la alternativa tardaba 1.4 segundos.

5. Los permisos de una credencial son parte de su configuración. El despliegue falló con `Cloud signing permission error` porque la clave de API se generó con rol *App Manager*. Subir compilaciones y crear activos de firma son permisos distintos: la firma en la nube requiere rol *Admin*.

6. Dos pipelines desplegando al mismo destino colisionan. El proyecto tenía Xcode Cloud conectado, y al añadir el despliegue en GitHub Actions ambos empezaron a publicar en TestFlight. App Store Connect rechaza un número de build repetido, y cada sistema lleva su propio contador, de modo que uno fallaba en cada push. No era un error que corregir sino un pipeline duplicado que retirar.

7. Un repositorio de código no debe vivir dentro de un servicio de sincronización. El proyecto estaba en una carpeta sincronizada con iCloud Drive, que genera copias de conflicto con sufijo " 2". Eso provocó una referencia de git corrupta y un archivo duplicado que rompió la compilación con un error de redeclaración. Se encontraron 24 archivos duplicados y el proyecto se movió fuera de la carpeta sincronizada.

8. Las herramientas de diagnóstico no deben quedarse encendidas. La aplicación se cerraba con SIGKILL al arrancar en un iPad. La causa era Address Sanitizer, activo en el esquema: instrumenta cada acceso a memoria y triplica el uso de memoria, lo que empujó el arranque más allá del límite del watchdog del sistema. Es una herramienta para cazar un error de memoria, no algo que dejar permanentemente activo.

9. Un fallo con un mensaje preciso vale el esfuerzo de escribirlo. El pipeline comprueba explícitamente sus precondiciones: que el secreto exista, que la clave decodificada sea un PEM válido, que el reporte de cobertura no esté vacío. Esas guardas detectaron un secreto vacío en el segundo paso del despliegue, en lugar de permitir que el proceso compilara diez minutos para morir con un error de firma ilegible.

10. Plan de mejora continua

10.1 Corto plazo (1 a 2 semanas)

#	ACCIÓN	MÉTRICA	ACTUAL	META
1	Corregir la generación de cuestionarios de 20 preguntas mediante generación por lotes	Tasa de éxito en cuestionarios de 20 preguntas	0 %	≥ 95 %
2	Extraer un protocolo de AIService para sustituirlo por un doble de prueba	Cobertura de FlashcardsGenerator	11 %	≥ 70 %
3	Ampliar el alcance de cobertura exigida a los servicios de sincronización	Archivos con cobertura exigida	5	12
4	Automatizar la aplicación de migraciones desde el pipeline	Migraciones aplicadas manualmente	10 de 10	0 de 10

10.2 Mediano plazo (1 a 2 meses)

#	ACCIÓN	MÉTRICA	ACTUAL	META
5	Extraer la lógica pura a un paquete Swift independiente	Cobertura medible por target	no aplicable	≥ 80 %
6	Añadir pruebas de integración para la sincronización	Pruebas de integración	0	≥ 15
7	Suite automatizada que verifique las políticas RLS intentando leer datos de otro usuario	Políticas verificadas automáticamente	0	100 %
8	Entorno de Supabase de pruebas desplegado desde el pipeline	Entornos	1	2
9	Especificación OpenAPI de las edge functions y zap-api-scan autenticado	Endpoints alcanzados por el escaneo	4	≥ 20

10.3 Largo plazo (6 meses)

#	ACCIÓN	MÉTRICA	ACTUAL	META
10	Telemetría de errores en producción	Tiempo medio de detección de un defecto	semanas	< 48 h
11	Reducir el tiempo de generación de un cuestionario	Tiempo hasta la primera pregunta	~30 s	< 8 s
12	Versión nativa de macOS, más allá de "Designed for iPad"	Plataformas con compilación propia	1	2

10.4 Propuestas de innovación tecnológica

Repetición espaciada con predicción de olvido. La base de datos ya almacena cada respuesta con su calificación y su página de origen, y el modo focus recoge una autoevaluación en la escala 0–5 del algoritmo SM-2. Con esos datos es posible calcular para cada concepto la fecha óptima de repaso y notificar al estudiante justo antes de que la curva de olvido lo alcance. La infraestructura de datos ya existe; falta el modelo de planificación.

Detección automática de temas débiles. Agrupando las preguntas falladas por página de origen y por concepto, el sistema puede identificar qué temas concentran los errores de un estudiante y generar automáticamente cuestionarios dirigidos a esas debilidades, en lugar de muestrear las páginas de forma uniforme.

Generación incremental por streaming. Hoy el cuestionario se genera en una sola respuesta del modelo, lo que provoca el fallo en cuestionarios largos y obliga a esperar a que termine. Generarlas de forma incremental permitiría empezar a estudiar con la primera pregunta lista, eliminar el truncamiento como modo de fallo y reducir el tiempo de respuesta percibido.

Reconocimiento de estructura en los apuntes. Aplicar análisis de la escritura manuscrita para identificar automáticamente títulos, definiciones, fórmulas y diagramas dentro de una página, y usar esa estructura tanto para organizar los cuadernos por temas sin intervención del usuario como para mejorar la calidad de las preguntas generadas.

11. Conclusiones

El módulo de flashcards se entregó funcional y distribuido, con las tres modalidades de estudio previstas, un CRUD completo sobre las preguntas y autenticación con roles diferenciados respaldada en la base de datos y no solo en la interfaz.

Alrededor del módulo se construyó la infraestructura que la actividad exige: 115 pruebas unitarias automatizadas donde antes no existía ninguna, un pipeline que integra, construye y despliega sin intervención manual, y análisis automatizado de calidad y seguridad cuyos hallazgos fueron corregidos y verificados.

Las lecciones más útiles no vinieron del código del módulo sino de la infraestructura: un trabajo en verde que no hacía nada, una herramienta de diagnóstico que impedía arrancar la aplicación, y dos pipelines compitiendo por el mismo destino. Todas comparten la misma raíz —dar por supuesto el estado de un sistema en lugar de comprobarlo— y son el insumo principal del plan de mejora continua.

12. Evidencias

ELEMENTO	UBICACIÓN
Repositorio	https://github.com/no-c-123/GirokIQ-iOS

ELEMENTO	UBICACIÓN
Módulo de flashcards	GirokIQ-ios/Features/Flashcards/
Panel de administración	GirokIQ-ios/Features/Admin/
Pruebas unitarias	GirokIQ-iosTests/
Pipeline CI/CD	.github/workflows/ci.yml
Workflow de seguridad	.github/workflows/security-scan.yml
Configuración de SonarQube	sonar-project.properties
Alcance de la cobertura	Scripts/coverage_targets.json
Reporte de pruebas	reports/pruebas-unitarias/
Reporte de cobertura	reports/cobertura/
Reporte de calidad	reports/calidad/sonarqube.md
Reportes de seguridad	reports/seguridad/
Diseño de roles	Docs/Roles.md
Procedimiento de despliegue	Docs/Despliegue.md
Migraciones de base de datos	supabase/migrations/
Panel de SonarQube	https://sonarcloud.io/dashboard?id=no-c-123_GirokIQ-iOS

[CAPTURA 13 — Estructura del repositorio mostrando la carpeta reports/ con los cuatro tipos de reporte]